Aplicația de pontaj — tehnologii și decizii

Oct 23, 2026 · @Ioan Tuns

1. Ce face aplicația

Aplicația este un program desktop portabil pentru Windows (un singur `.exe`, fără instalare), pentru pontajul lunar cu muncă în schimburi și tipărirea rapoartelor.

- Gestionează ture de 7 h, ture de 12 h, ture de noapte și gărzi de 24 h.
- Produce centralizatorul lunar după modelul folosit acum, plus exporturi pentru salarizare.
- Este un proiect separat de `salveazaclipa`.
- Nume propus: **Pontix**, cu Cadran ca alternativă. Cronos se evită din cauza confuziei cu Kronos (UKG). Disponibilitatea numelui (OSIM, EUIPO, domeniu) este de verificat.

2. Stack-ul ales

Stack-ul ales este Wails v2 cu Go și React, plus SQLite într-un singur fișier lângă executabil.

Strat	Tehnologie	Motiv
Aplicație desktop	Wails v2 (Go)	un `.exe` mic (~10–15 MB), folosește WebView2 din Windows 11
Partea din spate	Go (ales în locul Rust/Tauri)	simplu, compilează rapid, pachetul `time` gestionează bine fusul orar, biblioteci mature pentru rapoarte
Interfață	React + TypeScript	grilă editabilă de tip Excel (AG Grid Community sau TanStack Table)
Bază de date	SQLite prin `modernc.org/sqlite`	un fișier `.db` lângă `.exe`, Go pur, fără CGO
Acces la date	`sqlc`	cod Go cu tipuri, generat din fișiere SQL
Migrări	`goose` + `PRAGMA user_version`	un `.db` vechi se actualizează automat
Rapoarte PDF	`maroto` v2	aspect identic pe orice calculator, A4/A3 peisaj

Export Excel	`excelize`	același aspect ca centralizatorul actual; poate și importa date
Configurare	`config.toml` lângă `.exe`	adresa API, setări, profilurile LLM (mai târziu)

Variante respinse: Electron (prea mare), Tauri/Rust (mai complex, fără un beneficiu real aici), PostgreSQL/MySQL (au nevoie de server), fișiere JSON/CSV (fără relații și fără siguranța datelor).

3. Reguli pentru portabilitate

Toate datele și setările stau lângă executabil, ca aplicația să poată fi copiată sau mutată într-un singur folder.

- Datele și setările se salvează lângă `.exe`, nu în registru și nu în `%APPDATA%`.
- Aplicația nu cere drepturi de administrator și nu instalează servicii.
- Toate dependențele sunt incluse; ideal, executabilul e semnat digital.
- SQLite pornește cu `PRAGMA journal_mode=WAL` și `PRAGMA foreign_keys=ON`.
- Backup prin copierea fișierului `.db` sau cu `VACUUM INTO`.
- Limită: SQLite nu e potrivit dacă mai mulți utilizatori lucrează simultan pe același fișier de pe un disc din rețea.

4. Modelul de date

Orele lucrate se salvează ca interval cu dată și oră, iar absențele ca un cod pe zi. Totalurile se calculează, nu se introduc.

Tabel	Conținut
`angajati`	nume, marcă, funcție, normă ore/zi, activ
`tipuri_schimb`	cod, denumire, ora de start, ora de sfârșit, pauză (de ex. S1 06–14, S3 22–06)
`planificare`	ce s-a programat: angajat, dată, tip de schimb
`pontaj`	ce s-a lucrat efectiv: angajat, start, sfârșit, pauză, cod de absență, observații
`coduri_absenta`	C, CM, CEV, FP, CFS etc.

`sarbatori`	data, denumirea
`luni_inchise`	an, lună, închis la, închis de
`jurnal_modificari`	tabel, rând, câmp, valoare veche, valoare nouă, când, cine
`seturi_reguli`, `reguli_interval`, `categorii_ore`	regulile de tarif (secțiunea 5)
`coloane_raport`	coloanele de totaluri din centralizator și sursa lor
`pontaj_segmente`	rezultatul calculului pe categorii de tarif

Principii:

- O tură de noapte se introduce o singură dată. Tipul ei este normal, tură, noapte sau gardă.
- Planificatul și realizatul se păstrează separat.
- Coloanele de totaluri se definesc din setări, nu sunt scrise în cod.
- Închiderea lunii blochează modificările și îngheață calculul. Redeschiderea rămâne în jurnal.
- Pregătire pentru sincronizare: `id` UUID, `created_at`, `updated_at`, `deleted_at`, `owner_id`, `remote_id`, `sync_state`.

5. Calculul orelor și al tarifelor

Fiecare minut lucrat intră într-o singură categorie de tarif (categorii exclusive). Tipul zilei se ia din ziua calendaristică a fiecărui segment: până la 24:00 se aplică tariful zilei în curs, de la 00:00 tariful zilei următoare.

Regula de bază

- Pentru fiecare tip de zi, intervalele acoperă exact 00:00–24:00, fără goluri și fără suprapuneri. Câmpul `prioritate` a fost eliminat.
- Aplicația verifică setul de reguli la salvare și respinge un tip de zi cu un gol sau o suprapunere.

Tabelul de reguli (procentele marcate cu ? sunt încă de completat)

Tip zi	Interval	Categorie	Procent
LUCRĂTOARE	00:00–06:00	NOAPTE	?

LUCRĂTOARE	06:00–22:00	ZI	100
LUCRĂTOARE	22:00–24:00	NOAPTE	?
SÂMBĂTĂ	00:00–24:00	WEEKEND	200
DUMINICĂ	00:00–24:00	WEEKEND	200
SĂRBĂTOARE	00:00–24:00	SĂRBĂTOARE	?

Dacă în weekend se face diferență între zi și noapte, rândurile de sâmbătă și duminică se împart ca la zilele lucrătoare (00–06, 06–22, 22–24), cu procentele lor.

Exemplu: vineri 19:00 → sâmbătă 07:00

Interval	Zi	Categorie	Ore
19:00–22:00	vineri	ZI 100%	3
22:00–24:00	vineri	NOAPTE ?%	2
00:00–07:00	sâmbătă	WEEKEND 200%	7

Exemplu: duminică 19:00 → luni 07:00

Interval	Zi	Categorie	Ore
19:00–24:00	duminică	WEEKEND 200%	5
00:00–06:00	luni	NOAPTE ?%	6
06:00–07:00	luni	ZI 100%	1

O gardă de 24 h, sâmbătă 07:00 → duminică 07:00, intră în întregime la WEEKEND 200%.

Schema regulilor

```
seturi_reguli (
  id, denumire,
  valabil_de_la, valabil_pana_la      -- reguli versionate în timp
)

reguli_interval (
  id, set_id,
  tip_zi,                 -- LUCRATOARE / SAMBATA / DUMINICA / SARBATOARE
  ora_start, ora_sfarsit, -- acoperă exact 00:00–24:00 pe fiecare tip_zi
  categorie_cod
)
```

```
categorii_ore (
  cod, denumire,
  procent,                -- 100 / 125 / 200 ...
  coloana_raport          -- coloana din centralizator
)

pontaj_segmente (         -- rezultatul calculului; înghețat la închiderea
lunii
  pontaj_id, data_calendaristica, start, sfarsit,
  tip_zi, categorie_cod, minute
)
```

Calculul

- Funcția `Segmenteaza` (Go) taie fiecare tură la miezul nopții și la limitele intervalelor, apoi atribuie fiecărui segment categoria regulii care îl acoperă.
- Fusul orar este `Europe/Bucharest`, ca în nopțile cu schimbarea orei tura să aibă corect 11 sau 13 ore.
- Regulile sunt versionate, ca lunile vechi să se recalculeze cu regulile valabile atunci.
- Teste unitare obligatorii: vineri spre sâmbătă, duminică spre luni, gărzi de 24 h, sărbători, schimbarea orei, ture care se termină exact la o limită de interval.

6. Interfața și rapoartele

Interfața principală este o grilă angajați × zile, la fel de rapidă ca Excel. Rapoartele se tipăresc prin PDF generat, nu prin `window.print()`.

- Editare din tastatură cu coduri scurte: `7`, `12`, `N12`, `G24`, `C`, `CM`. Copiere și completare pe intervale.
- Weekendurile și sărbătorile sunt marcate automat. Totalurile se recalculează pe loc.
- Un clic pe celulă arată segmentele și categoriile de tarif.

Rapoartele inițiale:

1. Centralizatorul lunar (A4 sau A3 peisaj, câte o pagină pe secție, cu antetul repetat).
2. Fișa individuală pe angajat și lună.
3. Graficul de schimburi.
4. Exportul pentru salarizare, în Excel.

7. Etape viitoare (amânate)

Autentificarea GraphQL și asistentul LLM vin după prima versiune. Aplicația se pregătește de acum pentru ele, ca să nu fie nevoie de rescriere.

Autentificare GraphQL (de exemplu contul Saleor)

- Apelurile de rețea se fac doar din Go: `genqlient` și `net/http`.
- Login cu `tokenCreate`, reînnoire automată cu `tokenRefresh`.
- Tokenurile se păstrează în Windows Credential Manager (`go-keyring`), nu în SQLite sau în fișiere.
- Parola nu se salvează niciodată. Tokenul aplicației (`SALEOR_APP_TOKEN`) nu se include în `.exe`.
- Pregătire de acum: funcția `current_user()`, adresa API-ului în `config.toml`, coloanele de sincronizare.

Asistent LLM local (opțional)

- Rol: introducerea comenzilor în limbaj natural. Modelul nu calculează ore sau tarife.
- Rulare cu `llama-server` (llama.cpp) și un fișier `.gguf` lângă aplicație, ascultând doar pe `127.0.0.1`.
- Schimbarea modelului se face printr-un profil în `config.toml` și interfața Go `Asistent`, validată cu un set de 30–50 de comenzi reale.

Model	Licență	Mărime (Q4)	Rol
Qwen3 8B	Apache 2.0	~5 GB	principal, pentru calculatoare cu 16 GB RAM
Qwen3 4B	Apache 2.0	~2,5 GB	pentru calculatoare slabe (8 GB RAM)
Phi-4-mini	MIT	~2,5 GB	de testat; româna e punctul slab
Gemma 3 12B	licența Gemma (Google)	~7–8 GB	de testat; română mai bună, cu restricții de licență

Reguli pentru asistent:

- Formatul JSON e impus prin schemă.
- Datele, numele și calculele se rezolvă în Go.
- Modelul are voie să răspundă „neclar”.
- Utilizatorul confirmă fiecare modificare.
- Fără SQL generat de model; fiecare acțiune intră în jurnal.

8. Întrebări deschise

Acestea trebuie lămurite înainte de schema finală și de regulile de calcul.

- [ ] Tarifele concrete: procentul pentru noapte în zilele lucrătoare (22–24 și 00–06 sunt separate?), intervalul 06–07, sărbătorile, ajunul sărbătorilor.
- [ ] Gărzile: regula pentru coloana „Ore gardă 200%”.
- [ ] Ce înseamnă coloanele T, ZNL, L/FIN, celulele roșii și chenarele din centralizator.
- [ ] Afișarea turelor de noapte: împărțite la miezul nopții (5 + 7) sau trecute în ziua de început (12)?
- [ ] Volumul: numărul de angajați și de secții; se planifică lunar sau se completează doar ce s-a lucrat?
- [ ] Cum se scade pauza din segmentele unei ture.
- [ ] Validarea regulilor și a formatului rapoartelor cu contabilul sau responsabilul HR.
- [ ] Disponibilitatea numelui Pontix (OSIM, EUIPO, domeniu).

9. Ordinea de implementare

Prima versiune utilizabilă înseamnă pașii 1–4. Restul se adaugă peste ea, fără rescriere.

1. Datele de bază: angajați, tipuri de schimb, sărbători, coduri de absență.
2. Grila lunară de pontaj.
3. Motorul de calcul (`Segmenteaza`) și testele lui.
4. Centralizatorul în PDF și exportul Excel.
5. Planificarea și raportul planificat vs. realizat.
6. Închiderea lunii și jurnalul de modificări.
7. Mai târziu: autentificare și sincronizare, apoi asistentul LLM.